



ITES
RENÉ DESCARTES

Carrera:
Programación y WebMaster

Docente:
Mario Alonso Segovia Gutiérrez

Materia:
Proyecto de una Plataforma Virtual

Alumno(s):
Rodrigo Alejandro Barbosa Vidal

Fecha:
2
20/07/26

Parte 1. Identificación del problema

Problema	¿Qué ocurre?	¿Por qué es una mala práctica?	Consecuencias
1. Clases demasiado grandes	Existen clases con más de 1,800 líneas de código.	Una clase debe tener una única responsabilidad.	Dificultan el mantenimiento, las pruebas y la comprensión del código.
2. Métodos excesivamente largos	Algunas funciones tienen más de 250 líneas.	Los métodos deben realizar una sola tarea específica.	Aumenta la complejidad y la posibilidad de errores.
3. Código duplicado	Se reutiliza código mediante copiar y pegar.	Viola el principio DRY (Don't Repeat Yourself).	Los cambios deben realizarse en múltiples lugares, aumentando errores.
4. Variables poco descriptivas	Se utilizan nombres como x, dato, temp y obj.	Reduce la legibilidad y comprensión del código.	Incrementa el tiempo necesario para entender y mantener el sistema.
5. Clases con múltiples responsabilidades	Un archivo valida datos, consulta la base de datos, genera reportes y envía correos.	Mezcla responsabilidades que deberían estar separadas.	Cualquier modificación puede afectar varias funcionalidades.
6. Organización inconsistente	Cada desarrollador organiza el proyecto de forma diferente.	No existe una arquitectura uniforme.	El proyecto resulta difícil de mantener y escalar.
7. Ausencia de guía de estilos	No existen reglas de programación comunes.	Provoca código inconsistente entre desarrolladores.	Disminuye la productividad y aumenta la dificultad para colaborar.

8. Comentarios innecesarios	Los comentarios describen cada línea de código.	El código debe ser autoexplicativo.	Los comentarios se desactualizan rápidamente y generan confusión.
9. Falta de criterios de finalización	Las funciones se consideran listas solo porque "parecen funcionar".	No existe una validación objetiva de calidad.	Se integran errores al sistema.
10. No existen revisiones de código	Nunca se realizan revisiones entre desarrolladores.	Se pierden oportunidades para detectar errores y mejorar el código.	Aumentan los defectos en producción.
11. Alto acoplamiento	Cada nueva funcionalidad requiere modificar varias clases.	El sistema depende excesivamente entre sus componentes.	Se vuelve difícil agregar nuevas funciones sin romper las existentes.
12. Desarrollo de funciones innecesarias	Se implementan características no solicitadas por el cliente.	Contradice el principio YAGNI (You Aren't Gonna Need It).	Se desperdician tiempo, dinero y recursos.
13. Errores conocidos sin corregir	Se dejan fallas pendientes para avanzar más rápido.	Genera deuda técnica.	Los problemas aumentan con el tiempo y resultan más costosos de corregir.
14. Documentación desactualizada	La documentación nunca se actualiza.	La documentación debe reflejar el estado real del proyecto.	Dificulta el mantenimiento y la incorporación de nuevos desarrolladores.
15. Sin criterios de integración	No existe un proceso formal antes de integrar cambios al proyecto principal.	Se omiten controles de calidad.	Incrementa la posibilidad de introducir errores críticos.

Parte 2. Análisis desde Clean Code

1. Clases demasiado grandes

Principio incumplido:

Las clases deben tener una única responsabilidad y ser pequeñas.

Corrección:

Dividir la clase en varias clases especializadas según sus funciones.

Beneficios:

Mayor facilidad de mantenimiento.

Código más reutilizable.

Menor complejidad.

2. Métodos demasiado largos

Principio incumplido:

Los métodos deben ser cortos y realizar una sola tarea.

Corrección:

Extraer bloques de código en métodos independientes.

Beneficios:

Mayor claridad.

Más facilidad para realizar pruebas.

Menor probabilidad de errores.

3. Variables con nombres poco descriptivos

Principio incumplido:

Utilizar nombres significativos.

Corrección:

Cambiar nombres como *x* o *dato* por nombres que describan su propósito, por ejemplo:

nombreCliente

totalPago

fechaReserva

Beneficios:

El código se entiende sin necesidad de comentarios.

Reduce errores de interpretación.

4. Código duplicado**Principio incumplido:**

Evitar duplicación de código (DRY).

Corrección:

Crear funciones reutilizables o clases compartidas.

Beneficios:

Menor mantenimiento.

Menor posibilidad de inconsistencias.

Código más limpio.

5. Comentarios innecesarios**Principio incumplido:**

El código debe ser autoexplicativo.

Corrección:

Escribir código claro utilizando buenos nombres y métodos pequeños.

Beneficios:

Menos documentación obsoleta.

Mayor legibilidad.

Facilita futuras modificaciones.

Parte 3. Aplicación de los principios SOLID

Principio S — Single Responsibility Principle (SRP)

¿En qué consiste?

Cada clase debe tener una sola responsabilidad.

Evidencia

Un mismo archivo valida información, consulta la base de datos, genera reportes y envía correos.

Cambios:

Separar estas tareas en clases independientes:

Validador

Repositorio

Generador de reportes

Servicio de correo

Beneficios:

Menor complejidad.

Mayor reutilización.

Mantenimiento más sencillo.

Principio O — Open/Closed Principle (OCP)

¿En qué consiste?

El software debe estar abierto para extenderse pero cerrado para modificarse.

Evidencia

Cada nueva funcionalidad obliga a modificar varias clases existentes.

Cambios

Utilizar interfaces, herencia y composición para agregar funcionalidades sin modificar código ya probado.

Beneficios

Menor riesgo de introducir errores.

Desarrollo más rápido.

Mayor escalabilidad.

Principio D — Dependency Inversion Principle (DIP)

¿En qué consiste?

Las clases deben depender de abstracciones y no de implementaciones concretas.

Evidencia

Las clases realizan directamente consultas, validaciones y envío de correos.

Cambios

Implementar interfaces e inyección de dependencias.

Beneficios

Facilita pruebas unitarias.

Reduce el acoplamiento.

Mayor flexibilidad.

Principio I — Interface Segregation Principle (ISP)

¿En qué consiste?

Las interfaces deben contener únicamente los métodos que realmente necesitan los clientes.

Evidencia

Las clases realizan múltiples funciones relacionadas con diferentes módulos.

Cambios

Crear interfaces pequeñas y especializadas.

Beneficios

Código más organizado.

Menor dependencia entre módulos.

Parte 4. Deuda Técnica

Decisiones que generan deuda técnica

Código duplicado, Errores conocidos sin corregir, Falta de revisiones de código, documentación desactualizada, Funcionalidades innecesarias, Clases excesivamente grandes, Ausencia de estándares de programación.

¿Qué tipo de deuda técnica existe?

Principalmente deuda técnica de código, arquitectura, documentación y procesos.

¿Cómo afecta al mantenimiento?

Cada cambio requiere más tiempo, aumenta el riesgo de introducir nuevos errores y hace más difícil comprender el funcionamiento del sistema.

¿Qué impacto tendrá en tiempo y costo?

El tiempo de desarrollo crecerá porque será necesario corregir errores continuamente. Además, aumentarán los costos de mantenimiento, pruebas y capacitación de nuevos desarrolladores.

Acciones para disminuir la deuda, Refactorizar el código, Eliminar duplicidad, Aplicar principios SOLID, Implementar revisiones de código, Mantener la documentación actualizada, Establecer estándares de programación, Corregir errores antes de desarrollar nuevas funcionalidades.

Parte 5. Producto Mínimo Viable (MVP)

Funcionalidades del MVP

Funcionalidad	Justificación
----------------------	----------------------

Gestión de clientes	Es indispensable para administrar el gimnasio.
Registro de pagos	Permite controlar los ingresos del negocio.
Control de asistencia	Facilita el seguimiento de los clientes activos.
Reservación de clases	Es una función principal del servicio.
Reportes básicos de ingresos	Permite conocer el estado financiero del gimnasio.

Funcionalidades para futuras versiones

Funcionalidad	Justificación
Chat interno entre entrenadores	Aporta valor, pero no es indispensable para operar el gimnasio.
Integración con relojes inteligentes	Es una mejora tecnológica, no una necesidad inicial.
Sistema de recompensas	Puede aumentar la fidelización, pero no es prioritario.
Publicación automática en redes sociales	Es una función de marketing que puede implementarse posteriormente.
Tienda en línea de suplementos	Representa una nueva línea de negocio que puede desarrollarse en una versión futura.

Justificación general

El MVP debe enfocarse en las funciones esenciales que permitan administrar correctamente el gimnasio y generar ingresos. Las demás características agregan valor, pero pueden implementarse después sin afectar el funcionamiento principal del sistema.

Parte 6. Definition of Done (DoD)

Una funcionalidad se considerará terminada únicamente cuando cumpla los siguientes criterios:

1. Cumple completamente con los requisitos solicitados por el cliente.
2. El código sigue la guía de estilos del proyecto.
3. No contiene duplicación innecesaria de código.
4. Se respetan los principios SOLID y las buenas prácticas de Clean Code.
5. Todas las pruebas unitarias pasan correctamente.
6. La funcionalidad fue probada manualmente sin presentar errores.
7. La documentación técnica fue actualizada.
8. El código fue revisado y aprobado por otro desarrollador.
9. Se integró correctamente al repositorio principal sin conflictos.
10. No presenta errores críticos ni advertencias importantes.
11. Se actualizaron los casos de prueba correspondientes.
12. La funcionalidad fue validada por el responsable del proyecto antes de liberarse.

Parte 7. Reflexión Final

Considero que los principios SOLID y las prácticas de Clean Code son fundamentales para desarrollar software organizado, fácil de mantener y con menos errores. Ignorarlos puede generar deuda técnica, aumentar los costos de mantenimiento y retrasar el desarrollo del proyecto. En mi proyecto aplicaré estos conocimientos utilizando código claro, evitando la duplicación, asignando una única responsabilidad a cada clase y realizando pruebas antes de integrar nuevas funcionalidades.